

算术编码(编码 + 解码)

Huffman 给每个字符一段整数位的码字;算术编码则把整条消息压成 $[0,1)$ 里的一个小数。这是 15 分计算题的正主——老师标为重点,编码部分务必手到擒来。

直觉类比

把区间 $[0, 1)$ 想成一把没有刻度、可以无限细分的尺子。每读入一个符号,你就只保留尺子上属于它的那一小段,然后把这一小段当成新尺子继续切。消息越长,保留下来的区间就越窄;最后随便报一个落在这段区间里的小数,对方就能顺着同样的规则把整条消息还原出来。一个数,编码一整句话。

1. 核心概念

算术编码 (Arithmetic Coding)

一种无损熵编码。把整条消息视为一个整体,映射为半开区间 $[a, b) \subset [0, 1)$ 内的一个实数。通常比 Huffman 更接近熵,因为它不受"每字符至少 1 位"的整数约束。对照:Huffman 是"逐字符→整数位码字";算术是"整条消息→一个分数"。

概率区间 (Probability Range)

把 $[0, 1)$ 按各符号概率依次首尾相接地切成若干段,每个符号独占一段 $[Range_low, Range_high)$, 段宽 = 该符号概率。

累积概率 (Cumulative Probability)

某符号区间的下界 $Range_low$ = 排在它前面所有符号的概率之和;上界 $Range_high$ = 下界 + 自身概率。它决定了每个符号"占尺子的哪一段"。

下面这张表是所有计算的起点——先按固定顺序把符号概率累加成区间。注意必须有一个终止符(这里用 \$)来告诉解码器"消息到此结束"。

符号	概率 P	累积下界	区间 [RANGE_LOW, RANGE_HIGH)
A	0.4	0	[0, 0.4)
B	0.3	0.4	[0.4, 0.7)
C	0.2	0.7	[0.7, 0.9)
\$ (终止符)	0.1	0.9	[0.9, 1.0)

2. 编码的递推公式

维护三个量:当前区间下界 low、上界 high、以及区间宽度 $range = high - low$ 。初始 $low = 0, high = 1, range = 1$ 。每读入一个符号 s,把当前区间按 s 的比例缩窄:

缩窄公式(最需要背的两行)

`new_low = low + range × Range_low(s)`

`new_high = low + range × Range_high(s)`

然后更新 `range = new_high - new_low` 。

直觉:在当前这段尺子内部,再按符号 `s` 的比例位置切出一小段。读完终止符后,区间不再变化,最后输出任意满足 `low ≤ code < high` 的最短二进制小数即可。

例题 · 对 "ACB\$" 编码(15 分核心题型)

【题目】某信源有 4 个符号,概率为 $P(A)=0.4$ 、 $P(B)=0.3$ 、 $P(C)=0.2$ 、 $P(\$)=0.1$ ($\$$ 为终止符)。按此顺序累加得各符号的概率区间:

$$A \rightarrow [0, 0.4) \quad B \rightarrow [0.4, 0.7) \quad C \rightarrow [0.7, 0.9) \quad \$ \rightarrow [0.9, 1.0)$$

试对序列 A C B \$ 做算术编码:列出每读入一个符号后 low / high / range 的变化,给出最终区间,并求一个能落在该区间内的最短二进制小数作为码字。

【解】每一步都套那两行缩窄公式($\text{new_low} = \text{low} + \text{range} \times \text{Range_low}$, $\text{new_high} = \text{low} + \text{range} \times \text{Range_high}$):

- ① 初始化: $\text{low} = 0$, $\text{high} = 1$, $\text{range} = 1$ 。
- ② 读入 A, 区间 $[0, 0.4)$:
 $\text{low} = 0 + 1 \times 0 = 0$
 $\text{high} = 0 + 1 \times 0.4 = 0.4$
 $\text{range} = 0.4$ 。当前区间 $[0, 0.4)$ 。
- ③ 读入 C, 区间 $[0.7, 0.9)$:
 $\text{low} = 0 + 0.4 \times 0.7 = 0.28$
 $\text{high} = 0 + 0.4 \times 0.9 = 0.36$
 $\text{range} = 0.08$ 。当前区间 $[0.28, 0.36)$ 。
- ④ 读入 B, 区间 $[0.4, 0.7)$:
 $\text{low} = 0.28 + 0.08 \times 0.4 = 0.312$
 $\text{high} = 0.28 + 0.08 \times 0.7 = 0.336$
 $\text{range} = 0.024$ 。当前区间 $[0.312, 0.336)$ 。
- ⑤ 读入 \$(终止符), 区间 $[0.9, 1.0)$:
 $\text{low} = 0.312 + 0.024 \times 0.9 = 0.3336$
 $\text{high} = 0.312 + 0.024 \times 1.0 = 0.336$
 $\text{range} = 0.0024$ 。最终区间 $[0.3336, 0.336)$ 。

汇总成阅卷老师想看的那张表:

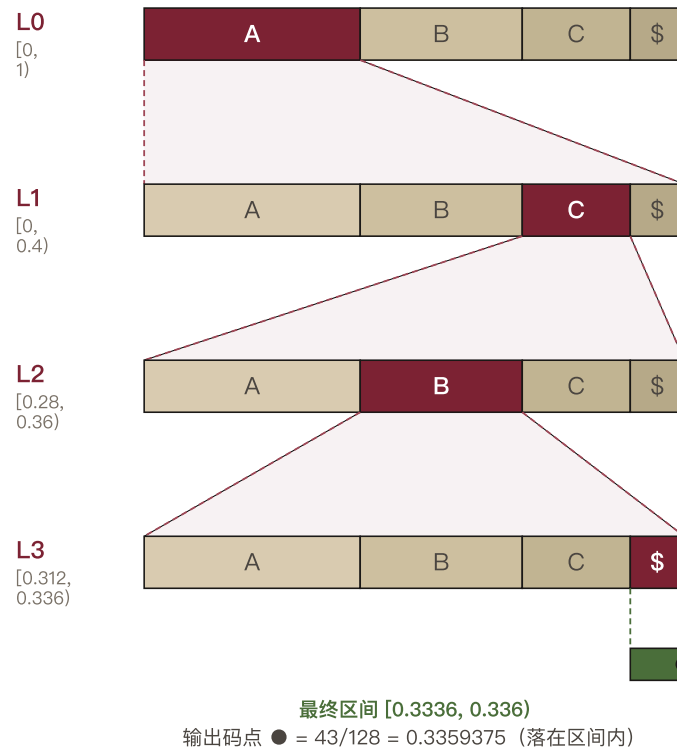
读入符号	LOW	HIGH	RANGE
(初始)	0	1.0	1.0
A	0	0.4	0.4
C	0.28	0.36	0.08
B	0.312	0.336	0.024
\$	0.3336	0.336	0.0024

找最短二进制码字:要在 $[0.3336, 0.336)$ 内找一个用最少小数位表示的二进制小数。逐位逼近——用 $k / 2^n$ 试:

$n=6$ 时步长 $1/64 \approx 0.0156$, 区间内需要 $0.3336 \times 64 \approx 21.35 \sim 0.336 \times 64 \approx 21.5$, 其间没有整数 $\rightarrow 6$ 位不够。

$n=7$ 时步长 $1/128 \approx 0.0078$, $0.3336 \times 128 \approx 42.7 \sim 0.336 \times 128 \approx 43.0$, 取整数 43 $\rightarrow 43/128 = 0.3359375$, 落在区间内。

结果:最终区间 $[0.3336, 0.336)$, 最短二进制码字 $= 43/128 = 0.0101011$ (即码流 0101011, 共 7 位)。验证: $0.0101011_2 = 1/4 + 1/16 + 1/64 + 1/128 = 0.3359375$, 确实 $0.3336 \leq 0.3359375 < 0.336$ ✓



编码：区间的递归细分。初始尺子 $[0, 1)$ 按 A·B·C·\$ 概率切成四段；读入一个符号就**只保留它那一段**，再把这段放大成新尺子（虚线梯形=“放大”）继续切。深红=当前所选符号段。四步 A→C→B→\$ 后区间缩到 $[0.3336, 0.336)$ ，任取其中一个最短二进制小数即为码字。怎么读：每往下一层，就是上一层深红段被拉满整条尺子。

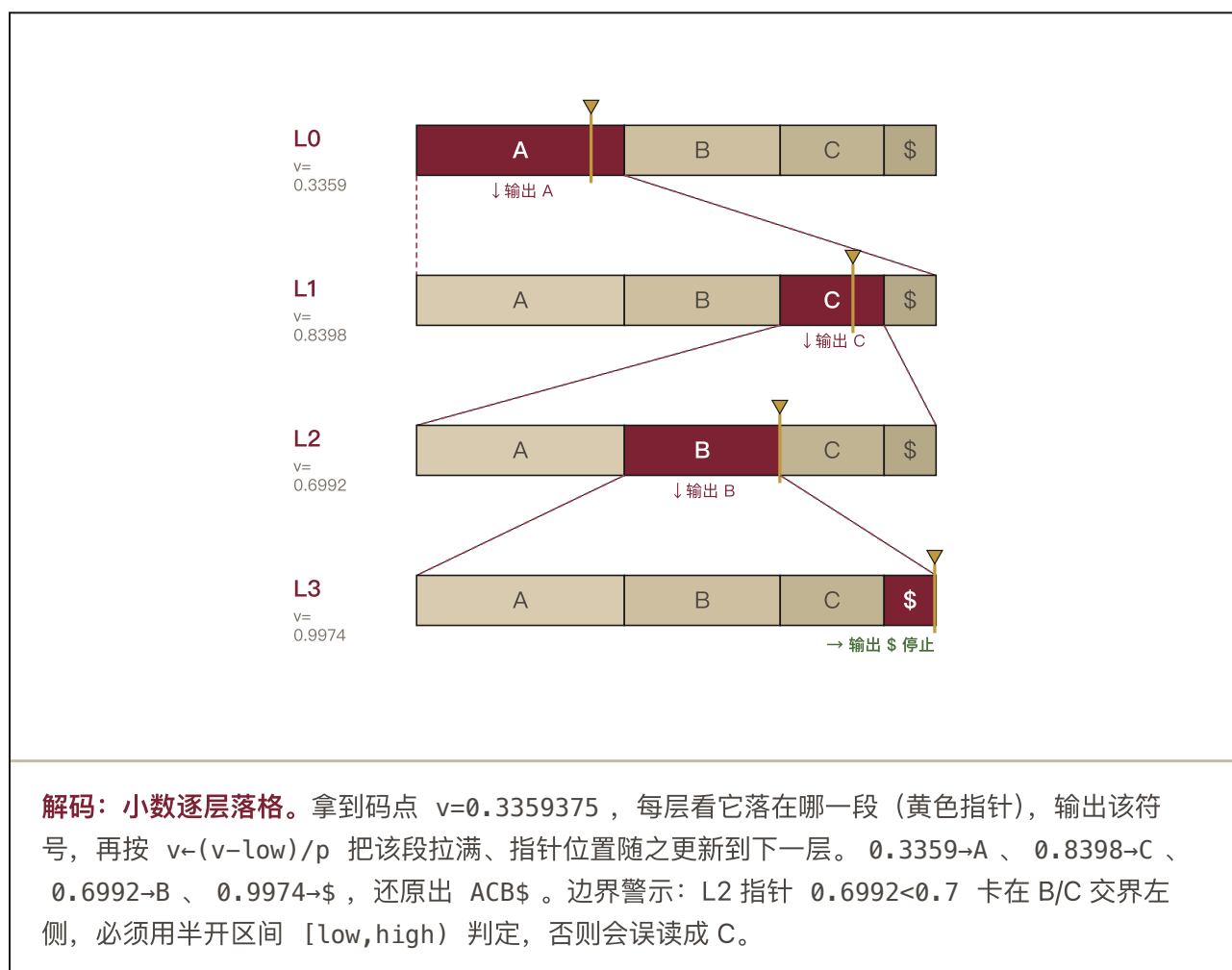
3. 解码:反着走一遍(了解即可)

老师说解压部分了解即可,但过一遍能帮你彻底看懂编码。解码器拿到小数 $value = 0.3359375$,用同一张概率表反向定位:每一步找出 $value$ 落在哪个符号区间,输出该符号,再把 $value$ "放大还原"回单位区间,公式:

$$value \leftarrow (value - Range_low(s)) / p(s)。$$

当前 VALUE	落在区间	输出	还原 $(VALUE - LOW) / P$
0.3359375	$[0, 0.4) \rightarrow A$	A	$(0.3359375 - 0) / 0.4 = 0.83984375$
0.83984375	$[0.7, 0.9) \rightarrow C$	C	$(0.83984375 - 0.7) / 0.2 = 0.69921875$
0.69921875	$[0.4, 0.7) \rightarrow B$	B	$(0.69921875 - 0.4) / 0.3 = 0.99739...$
0.99739...	$[0.9, 1.0) \rightarrow \$$	\$	终止符 $\rightarrow$ 停止

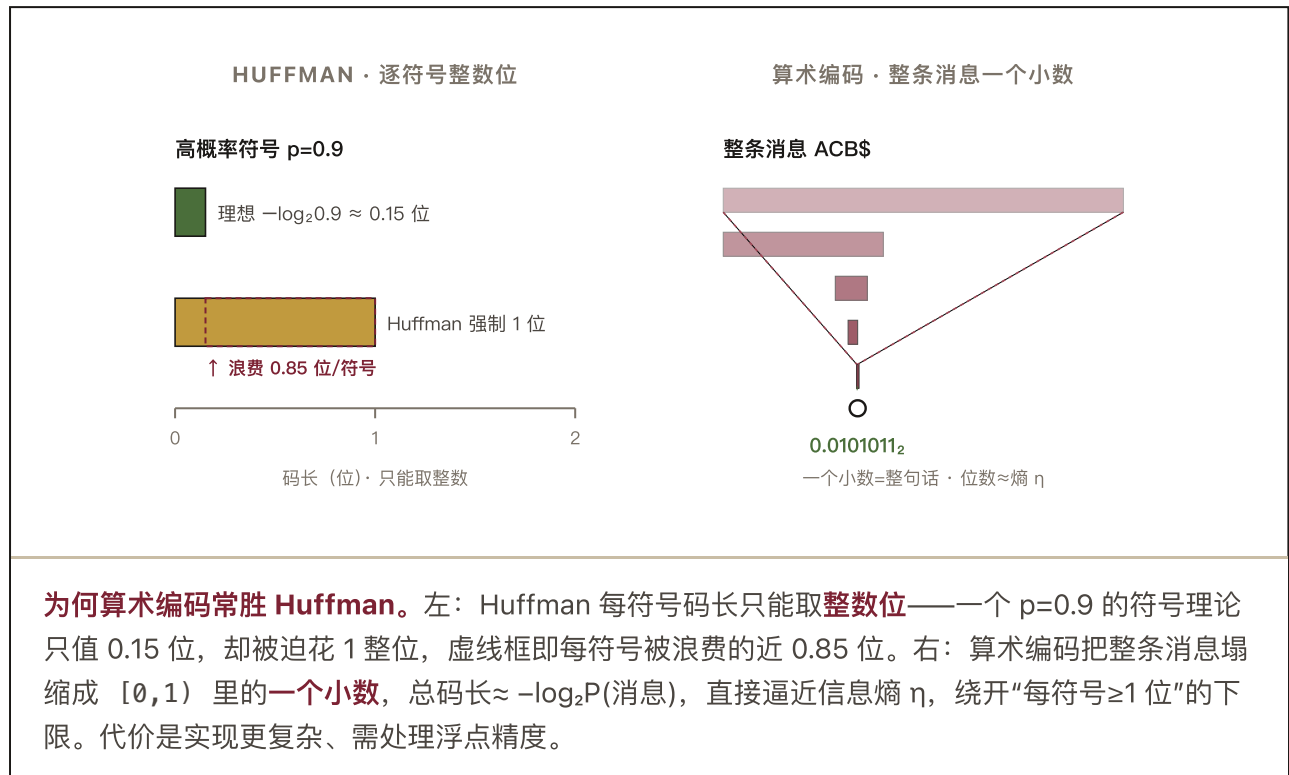
解出 A C B \$,与原消息完全一致 ✓。注意第三步 $0.699... < 0.7$,所以落在 B 而不是 C——这类"卡边界"判断是解码最容易出错的地方,务必用严格的半开区间 $[low, high)$ 判断。



4. 为什么它能赢 Huffman

Huffman 给每个符号分配整数位码字,当某符号概率很高(比如 0.9)时,理论上它只值 $-\log_2 0.9 \approx 0.15$ 位,可 Huffman 至少得给它 1 位——浪费。算术编码把整条消息合

成一个分数,能逼近熵 η , 不受整数位约束。代价是实现更复杂、要处理浮点精度。简答题里"为什么算术编码常优于 Huffman"就答这一句:它对整条消息编码、用分数码字逼近熵,绕开了每符号至少 1 位的下限。



自测题

沿用本节概率表(A[0,0.4)、B[0.4,0.7)、C[0.7,0.9)、S[0.9,1.0)),对序列 B A 编码,读完这两个符号后的当前区间 [low, high) 是?

- A. [0.4, 0.7)
- B. [0.28, 0.36)
- C. [0.4, 0.52)
- D. [0.4, 0.5)

▼ 答案与解释

答案:C [0.4, 0.52)

逐步套公式:

初始 low=0, high=1, range=1。

读入 B[0.4,0.7): $\text{low} = 0 + 1 \times 0.4 = 0.4$, $\text{high} = 0 + 1 \times 0.7 = 0.7$, $\text{range} = 0.3$ 。

读入 A[0,0.4): $\text{low} = 0.4 + 0.3 \times 0 = 0.4$, $\text{high} = 0.4 + 0.3 \times 0.4 = 0.52$, $\text{range} = 0.12$ 。

得 [0.4, 0.52)。

干扰项辨析:A 是只读入 B 后的区间(漏了第二步);B 是例题里 "AC" 的区间(记错序列);D 是把 0.3×0.4 误算成 0.1。关键别忘了第二步的宽度要乘上当前 **range** 而非 1。

下一步 → 无损压缩告一段落,进入有损压缩的度量:[0013 · 失真度量·PSNR·率失真理论](#),看"压得越狠、失真越大"这道权衡如何量化。